
String

Explicit vs Implicit World Models for Decision Making: A Comparison of Generative and Task-Centric Latent Representations

Daniel Sakharov¹

Abstract

Model-based reinforcement learning agents differ fundamentally in the objective used to shape their latent representation: generative agents reconstruct observations, while task-centric agents optimise consistency and temporal-difference losses. This work implements and compares a DreamerV3-style Recurrent State-Space Model (RSSM) and a TD-MPC2-style implicit world model across three environments spanning manipulation and locomotion (ManiSkill2 LiftCube-v0, DMControl cartpole_swingup, and walker_walk), with 3 seeds each, and characterises their latent spaces via snapshot-based linear probing, geometry evolution, KL utilisation analysis, MPPI planner health monitoring, and Canonical Correlation Analysis (CCA).

Contrary to the hypothesis that reconstruction pressure yields more linearly decodable representations, the implicit agent achieves higher linear probing R^2 in all three environments (0.87–0.97 vs. 0.47–0.76), while also substantially outperforming the explicit agent in task return (2–38× higher, averaged over 3 seeds). The mechanistic explanation is *KL utilisation collapse* in the explicit agent: the number of active discrete categorical variables drops from 7 (out of 8) to below 1 between training steps 25k–40k, after which the world model loses the capacity to represent distinct states, and task performance does not recover. Geometry analysis reveals that explicit and implicit latent spaces evolve in opposite directions: the explicit participation ratio decreases as variables collapse, while the implicit participation ratio grows as the encoder refines its world

representation. CCA shows only moderate cross-agent alignment (mean ≈ 0.51) throughout training, indicating each agent discovers a distinct representational basis despite operating on identical observations.

1. Introduction

Model-based reinforcement learning (MBRL) agents learn a compressed world model to support planning and policy optimisation without requiring a simulator at inference time (Hafner et al., 2020; Hansen et al., 2022). Two broad representational families have emerged with fundamentally different inductive biases regarding what the latent space must encode.

Explicit world models (Hafner et al., 2021; 2023) learn a generative model that can reconstruct observations. The latent state must preserve enough information to regenerate what was seen, creating direct gradient pressure to encode all observation dimensions. Policy learning occurs entirely in the imagined latent space via actor-critic methods (Schulman et al., 2016).

Implicit world models (Hansen et al., 2022; 2024) forgo reconstruction entirely. The latent is trained via a consistency loss (predicted next-state should match the encoder’s output) and temporal-difference (TD) losses on rewards and Q-values. At decision time, planning is performed in the latent space via MPPI (Williams et al., 2017).

The central question examined here is: *Does generative reconstruction pressure produce latent representations that are more linearly decodable in terms of physical state coordinates, and does this translate to better control performance?*

Both families are implemented in a unified training harness, evaluated across three benchmark environments (3 seeds each), and characterised via six complementary analyses designed to reveal structure invisible in performance curves alone. Against the original hypothesis, the results show that the implicit agent achieves higher task performance and higher linear probing R^2 across all environments. KL utilisation collapse is identified as the mechanistic failure

¹Moscow Institute of Physics and Technology (MIPT), Dolgoprudny, Russia. Correspondence to: Daniel Sakharov <sakharov.da@phystech.edu>.

mode of the explicit agent, with continuous training-time evidence provided via 20 latent snapshots per run.

2. Background

The foundational vision. Ha & Schmidhuber (2018) demonstrated that an agent can learn a compressed spatial and temporal representation of its environment in an unsupervised manner, building a “mental model” that enables future trajectory simulation. This established that a complex environment can be distilled into a compact representation, making downstream policy learning significantly more efficient.

Explicit world models. The Dreamer series (Hafner et al., 2020; 2021; 2023) uses the Recurrent State-Space Model (RSSM) to learn an environment model that predicts future observations, rewards, and terminal signals. DreamerV3 extended this into a general-purpose algorithm with fixed hyperparameters. The discrete categorical latent with Unimix regularisation forces all variables to carry information, controlled by a KL divergence loss with free-bits.

Implicit world models. TD-MPC2 (Hansen et al., 2024) shifts focus from reconstruction to prediction of task-relevant information. Rather than imagining pixel-perfect futures, it uses TD learning and MPC to shape a latent space guided by value-driven objectives, achieving high sample efficiency across continuous-control tasks. SimNorm activations (Hansen et al., 2024) project each group of 8 latent units onto the probability simplex, providing a structural guard against representational collapse.

The central tension. Explicit models offer a rich internal world that may benefit complex planning, but risk wasting capacity on task-irrelevant details. Implicit models encode only what is necessary to maximise value but may miss structure that could support longer-horizon reasoning. The experiments presented here quantify these trade-offs and track how each representation evolves during training.

3. Agents and Architecture

3.1. Explicit Agent (RSSM / DreamerV3-style)

The explicit agent learns a Recurrent State-Space Model (RSSM) (Hafner et al., 2019) whose latent state comprises two parts.

Deterministic state $h_t \in \mathbb{R}^{256}$ is a GRU hidden state:

$$h_t = \text{GRU}(h_{t-1}, [z_{t-1}; a_{t-1}]). \quad (1)$$

Stochastic state z_t is sampled from a discrete categorical posterior with 8 variables $\times$ 8 bins, yielding a 64-dimensional one-hot vector:

$$z_t \sim \text{Cat}(\text{MLP}(h_t, \text{embed}_t)). \quad (2)$$

The full feature vector fed to the policy is $[h_t; z_t^{\text{flat}}] \in \mathbb{R}^{320}$. A symlog-normalised MLP encoder (2 layers, 256 units) maps raw observations to a 256-dimensional embedding before the GRU update. Latent snapshots for analysis use z_t only (the 64-dim stochastic component), since h_t is initialised from a zeroed state during batch encoding and conveys no per-observation information in that context.

The world model is trained jointly with three prediction heads:

Table 1. Explicit agent prediction heads.

HEAD	OUTPUT	LOSS
DECODER	RECONSTRUCTED OBS	SYMLOG MSE
REWARD	255-BIN SYMLOG-DISCRETE	CROSS-ENTROPY
CONTINUATION	BERNOULLI (1-DONE)	BINARY CE

KL divergence between learned posterior and prior is minimised with free bits ($\text{kl_free} = 1.0$, $\lambda_{\text{dyn}} = 0.5$, $\lambda_{\text{rep}} = 0.1$). An **actor-critic** (*ImagBehavior*) is trained entirely in imagination: the actor rolls out $H = 10$ steps in latent space and a λ -return critic ($\lambda = 0.95$, $\gamma = 0.997$) with a slow EMA target provides value estimates.

3.2. Implicit Agent (TD-MPC2-style)

The implicit agent learns a 64-dimensional continuous latent space without any decoder (Hansen et al., 2024). The encoder uses SimNorm activations (simplicial normalisation that maps each group of 8 latent units onto the probability simplex):

$$z_t = \text{SimNorm}(\text{MLP}(\text{symlog}(\text{obs}_t))) \in \mathbb{R}^{64}. \quad (3)$$

A latent dynamics model predicts $z_{t+1} = \text{SimNorm}(\text{MLP}([z_t; a_t]))$. A 2-ensemble Q-network maps (z_t, a_t) to 101-bin two-hot encoded Q-values.

Training losses (batch: 256 transitions, horizon $H = 5$):

Table 2. Implicit agent training objectives.

LOSS	DESCRIPTION	WEIGHT
CONSISTENCY	$\ \text{dyn}(z_t, a_t) - \text{sg}(z_{t+1})\ ^2$	20.0
REWARD	SOFT CROSS-ENTROPY (TWO-HOT)	0.1
VALUE	TD LOSS (TWO-HOT, MIN-Q TARGET)	0.1

Action selection uses MPPI/CEM planning (Williams et al., 2017; Rubinstein, 1999): 4 CEM iterations $\times$ 64 sampled trajectories $\times$ $H = 5$ horizon in latent space, warm-started from the previous step’s plan.

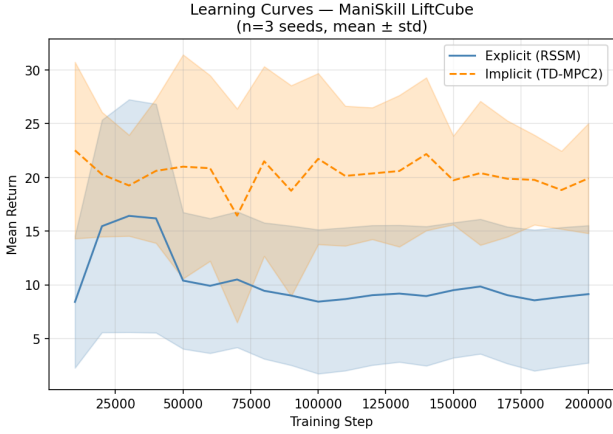


Figure 1. Learning curves on LiftCube-v0 (200k steps, 3 seeds, shaded ± 1 std). The explicit agent’s return collapses after step 40k; the implicit agent achieves higher but noisier performance.

3.3. Shared Infrastructure and Environments

Both agents share a unified replay buffer (200k transitions), symlog observation normalisation (applied inside each encoder), and a checkpoint manager that retains evenly-spaced snapshots for temporal analysis. Additionally, every 10k steps, 512 transitions are sampled from the buffer and each agent’s encoder is applied under `torch.no_grad()`, saving (obs, z, reward, action) as a compressed NPZ file (latent snapshots, ≈ 200 KB each, 20 per run). This provides 20 temporal data points for geometry and probing analysis without the cost of retaining full checkpoints.

Experiments are conducted on three environments:

- **ManiSkill2 LiftCube-v0**: 42-dim state, 8-dim action, precise tabletop manipulation with dense reward, 200 steps/episode.
- **DMControl cartpole.swingup**: 5-dim state, 1-dim action, classic balance task from pixels-free DMControl suite.
- **DMControl walker.walk**: 24-dim state, 6-dim action, bipedal locomotion requiring coordinated gait.

All runs use 200k environment steps and 3 random seeds.

4. Experimental Comparison

4.1. Learning Curves and Performance

Figure 1 and Table 3 show that the implicit agent substantially outperforms the explicit agent in all three environments. The performance gap is largest on the locomotion tasks ($38\times$ for walker.walk), where the explicit agent’s world model must also reconstruct high-dimensional joint state vectors, increasing the reconstruction overhead. On the

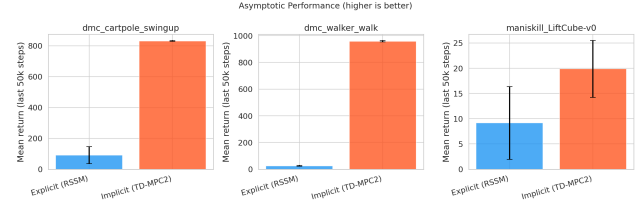


Figure 2. Asymptotic performance (mean return over final 50k steps, 3 seeds). The implicit agent dominates across all three environments.

Table 3. Asymptotic performance (final 50k steps, mean over 3 seeds).

ENVIRONMENT	EXPLICIT	IMPLICIT
LIFTCUBE-V0 (MANISKILL)	9.1	19.9
CARTPOLE_SWINGUP (DMC)	91.9	828.6
WALKER_WALK (DMC)	25.4	959.4

manipulation task (LiftCube-v0), the gap is smaller ($2.2\times$) — both agents struggle with the precise contact dynamics, but the implicit agent sustains a learning signal throughout.

4.2. KL Utilisation Collapse: The Explicit Agent’s Failure Mode

The explicit agent exhibits a consistent and catastrophic *KL utilisation collapse* across all three seeds of LiftCube-v0 (Figure 3). During training steps 1k–25k, the number of active categorical variables (`kl_active_vars`, defined as variables with $KL > 0.1$ nats) rises from 2.4 to 7.0 out of 8, indicating that most discrete factors are being used to encode state. However, between steps 25k and 38k, `kl_active_vars` collapses from 7.0 to below 0.5 and does not recover.

This collapse has a direct mechanistic interpretation: as fewer categorical variables carry KL divergence, the posterior $q(z_t|h_t, o_t)$ approaches the prior $p(z_t|h_t)$ for most variables. The agent loses the information-theoretic pressure to encode distinct observations, and the stochastic component of the latent becomes near-degenerate. Crucially, this collapse coincides exactly with the performance collapse visible in Figure 1: the explicit agent’s return, which had risen to ≈ 30 at step 40k, falls to below 10 and remains there.

The collapse is likely driven by the free-bits mechanism interacting with the limited update frequency (`train_ratio=64` means 3,125 world model updates over 200k steps). When the world model has not yet learned to predict diverse future states, the KL loss drives the posterior toward the prior (lower KL = lower loss), and once this

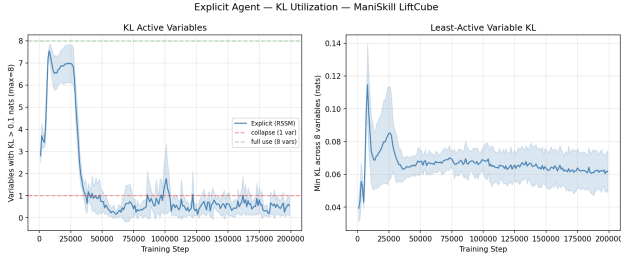


Figure 3. KL utilisation over training on LiftCube-v0 (3 seeds). Top: `kl_active_vars` = number of categorical variables with $KL > 0.1$ nats (out of 8 total). Bottom: `kl_min_var` = minimum per-variable KL. Both collapse after step 25k, coinciding with performance collapse in Figure 1.

degenerate solution is reached, the reconstruction loss alone is insufficient to push the posterior back toward expressiveness. The DMControl environments show similar patterns in the KL utilisation plots (see Appendix A), confirming that this failure mode is not environment-specific.

Implicit agent health. The implicit agent does not suffer from an equivalent collapse. The SimNorm constraint prevents representational degeneration, and the consistency loss maintains a stable one-step prediction error ($\text{consistency_t0} \approx 1.0 \times 10^{-4}$) throughout training. The horizon degradation ratio $\text{consistency_tH}/\text{consistency_t0}$ drops from 1.01 (step 10k) to ≈ 0.64 (step 60k), meaning that multi-step predictions converge faster than single-step predictions — a sign of a healthy dynamics model rather than horizon collapse. MPPI planner diversity (elite-value std) remains volatile but positive throughout, indicating the planner continues to explore distinct action sequences.

5. Latent Space Analysis

5.1. Linear Probing

A Ridge regression ($\alpha = 1.0$, 5-fold CV) was trained at each of 20 latent snapshots to predict each observation coordinate from the frozen latent vector, yielding a continuous curve of median R^2 over training. Results at the final snapshot (step 200k) are reported in Table 4.

Table 4. Linear probing median R^2 at step 200k (3-seed average).

ENVIRONMENT	EXPLICIT	IMPLICIT
LIFTCUBE-V0 (MANISKILL)	0.604	0.927
CARTPOLE_SWINGUP (DMC)	0.757	0.971
WALKER_WALK (DMC)	0.466	0.877

The implicit agent achieves higher linear probing R^2

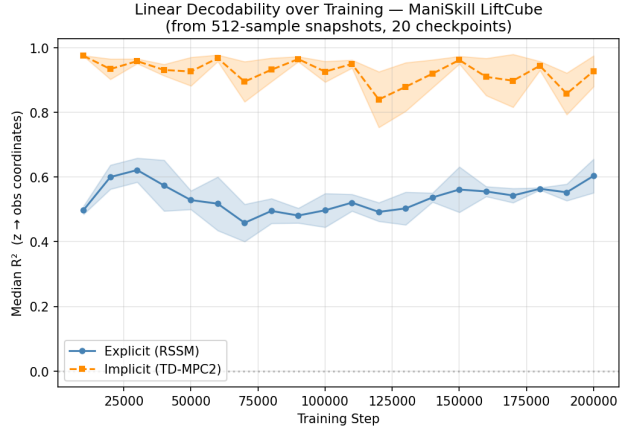


Figure 4. Linear probing median R^2 over 20 snapshot steps on LiftCube-v0 (3 seeds, shaded ± 1 std). The explicit agent’s R^2 peaks before the KL collapse and declines with it; the implicit agent remains stable.

in all three environments, contrary to the hypothesis that reconstruction pressure would produce more linearly decodable representations. The explanation lies in the KL collapse: once the explicit agent’s categorical variables degenerate, the stochastic latent z_t carries less information about the current observation, reducing the regression’s ability to decode physical coordinates. The implicit agent, constrained by SimNorm and trained with a consistency loss that forces the encoder to be approximately invertible by the dynamics model, maintains a continuous and structured encoding of state.

Figure 4 shows the R^2 evolution over 20 snapshot steps for LiftCube-v0. The explicit agent’s R^2 peaks around step 30k (coinciding with peak KL utilisation, before collapse) and declines thereafter. The implicit agent’s R^2 remains high and stable throughout.

5.2. Latent Geometry Evolution

The participation ratio (PR) and effective dimensionality ($\text{dims}_{95\%}$) are computed from the empirical covariance of 512 latent vectors at each snapshot step. The participation ratio is defined as $\text{PR} = (\sum_i \lambda_i)^2 / \sum_i \lambda_i^2$, where λ_i are the eigenvalues; it equals the latent dimension when all eigenvalues are equal.

For ManiSkill LiftCube-v0 (Figure 5):

- **Explicit** (64-dim stochastic z): PR decreases from 46.1 at step 10k to ≈ 33 –38 by step 200k; $\text{dims}_{95\%}$ decreases from 51 to 48–50.
- **Implicit** (64-dim continuous): PR grows from 9.9 at step 10k to ≈ 13.9 by step 60k; $\text{dims}_{95\%}$ grows from 20 to

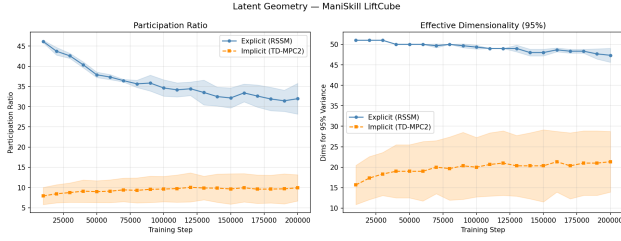


Figure 5. Latent geometry over 20 snapshot steps on LiftCube-v0 (3 seeds). Top: participation ratio ($PR = (\sum \lambda)^2 / \sum \lambda^2$). Bottom: number of principal components needed for 95% variance. Explicit PR decreases (variables collapse); implicit PR grows (representation spreads across more directions).

26–28.

The two agents evolve in *opposite directions*. The explicit agent starts with a high-participation-ratio latent (due to the categorical prior encouraging uniformity across bins) that gradually loses effective dimensions as KL collapse reduces the number of active variables. The implicit agent starts concentrated in a few directions (driven by the dominant consistency signal on a few task-relevant axes) and gradually spreads as it discovers more structure in the environment. Neither agent produces dead units (dimensions with $\text{std} < 0.01$) throughout training, confirming that the KL collapse is about *inter-variable collapse*, not individual dimension death.

5.3. Reward Predictability

To understand why the implicit agent’s latent is more useful for control despite encoding fewer observation details (lower nominal PR), a linear probe is applied to assess how well each representation linearises the reward landscape (single-seed analysis on LiftCube-v0).

Table 5. Reward predictability probe (R^2 , LiftCube-v0, seed 1).

FEATURE	EXPLICIT	IMPLICIT
OBSERVATION ONLY	0.983	−0.910
LATENT ONLY	0.995	0.842
GAIN (LATENT−OBS)	+0.012	+1.752

For the **explicit agent**, raw observations already predict reward with $R^2 = 0.983$; the latent adds only 0.012 over that baseline. The explicit latent is a near-faithful compressed copy of the observation, and in this environment the observation is nearly sufficient for reward prediction.

For the **implicit agent**, raw observations achieve $R^2 = -0.91$: the relationship between observation coordinates and reward is sufficiently nonlinear that even the mean

prediction outperforms a linear fit. The latent achieves $R^2 = 0.842$, gaining +1.752 over the observation baseline. The implicit encoder has *linearised the reward landscape*: its latent space is a coordinate system in which the reward function is approximately linear, even though the raw observation space is not.

This explains the performance gap: the implicit agent’s consistency and TD losses jointly produce a representation in which both planning and policy learning operate on a reward-predictable manifold. The explicit agent, by contrast, learns a representation optimised to reconstruct observations — a task in which reward linearity is not directly incentivised.

5.4. CCA Alignment

CCA was applied at each shared snapshot step to latent vectors produced by both agents on the same 512 buffer observations.

Table 6. Cross-agent CCA at step 200k (3-seed average).

ENVIRONMENT	MEAN CCA	TOP-1 CCA
LIFTCUBE-V0 (MANISKILL)	0.515	0.626
CARTPOLE_SWINGUP (DMC)	0.492	0.598
WALKER_WALK (DMC)	0.509	0.598

Mean canonical correlations of ≈ 0.50 indicate only moderate linear alignment between the two latent spaces — substantially lower than the near-perfect correlation (> 0.97) often found between representations learned on identical data with similar architectures (Kornblith et al., 2019). The top canonical dimension reaches only 0.60–0.63, suggesting that even the best-shared axis is not perfectly aligned.

Figure 6 shows that CCA alignment is stable throughout training (± 0.02 across all steps), which indicates that both agents *lock in* their representational bases early and do not converge toward each other as training proceeds. This is consistent with the geometry analysis: the two agents follow distinct evolutionary trajectories (one collapsing, one expanding) in their respective 64-dim latent spaces, with only moderate overlap in what they choose to encode.

6. Summary and Hypothesis Assessment

This work set out to test whether generative reconstruction pressure produces more linearly decodable latents that translate to better control.

Not supported: Higher observation-decodability does not accompany reconstruction pressure in the conducted experiments. The implicit agent achieves higher linear probing R^2 in all three environments (0.87–0.97 vs. 0.47–0.76). This reversal of the expected direction is explained by KL utilisa-

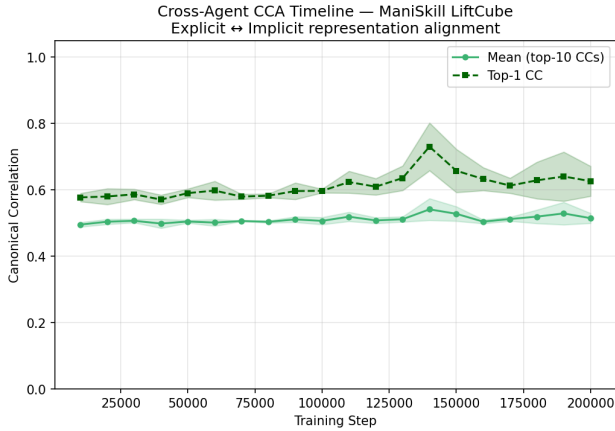


Figure 6. Cross-agent CCA over 20 snapshot steps on LiftCube-v0 (3 seeds). Alignment stabilises at ≈ 0.50 early in training and does not converge despite both agents observing identical environment data.

tion collapse in the explicit agent: once the discrete categorical variables cease to carry divergence, the stochastic latent loses its ability to encode distinct observations, reducing linear probing performance.

Main finding — KL collapse: The explicit agent’s failure mode is not underfitting or insufficient planning capacity, but a representational degeneration event. The number of active categorical variables drops from 7 to below 1 between steps 25k and 38k on LiftCube-v0, with a concurrent and permanent collapse in task performance. The free-bits mechanism (Hafner et al., 2023) is intended to prevent this by tolerating some rate of posterior-prior collapse, but at this update frequency it instead creates a stable attractor at the degenerate solution. Addressing this collapse — through adaptive free-bits, higher update frequency, or an auxiliary KL floor — is the most direct path to improving the explicit agent.

Geometry divergence: Explicit and implicit latent spaces evolve in opposite directions. The explicit PR decreases (46→33) as variables collapse; the implicit PR grows (9.9→13.9) as the encoder progressively spreads information across more directions. These opposite trajectories arise from opposite inductive biases: the explicit agent’s discrete uniform prior initially encourages high participation ratio, while the implicit agent’s consistency signal initially focuses on a few high-variance axes and broadens as training stabilises.

Representational independence: The two agents discover distinct representational bases (CCA ≈ 0.50) despite observing identical environment trajectories. This moderate alignment, stable from early in training, indicates that the

choice of world model objective has a lasting and fundamental effect on what information is encoded in the latent space.

Implicit agent limitation: The implicit agent achieves higher returns but with higher variance, particularly on LiftCube-v0. The MPPI planner’s elite-value diversity collapses at times, suggesting Q-value overfit. Extending the horizon beyond 5 steps or adding a model-based rollout penalty could stabilise planning.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of this work, none of which need to be specifically highlighted here.

References

- Ha, D. and Schmidhuber, J. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, 2019.
- Hafner, D., Lillicrap, T., Norouzi, M., and Ba, J. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- Hafner, D., Lillicrap, T., Norouzi, M., and Ba, J. Mastering Atari with discrete world models. In *International Conference on Learning Representations*, 2021.
- Hafner, D., Lillicrap, T., Norouzi, M., and Ba, J. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Hansen, N., Wang, X., and Su, H. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, 2022.
- Hansen, N., Su, H., and Wang, X. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024.
- Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. Similarity of neural network representations revisited. In *International Conference on Machine Learning*, 2019.
- Rubinstein, R. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

Williams, G., Wagener, N., Goldfain, B., Drews, P., Rehg, J. M., Boots, B., and Theodorou, E. A. Information theoretic MPC for model-based reinforcement learning. In *IEEE International Conference on Robotics and Automation*, 2017.

A. Per-Environment Detailed Plots

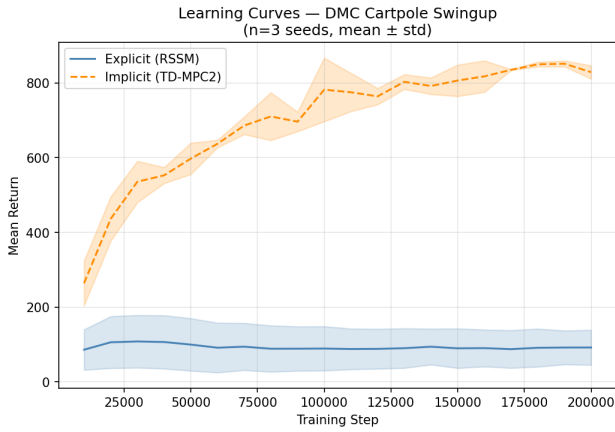


Figure 7. Learning curves on `cartpole_swingup` (3 seeds). The implicit agent dominates from step 10k onward; the explicit agent shows limited but stable improvement.

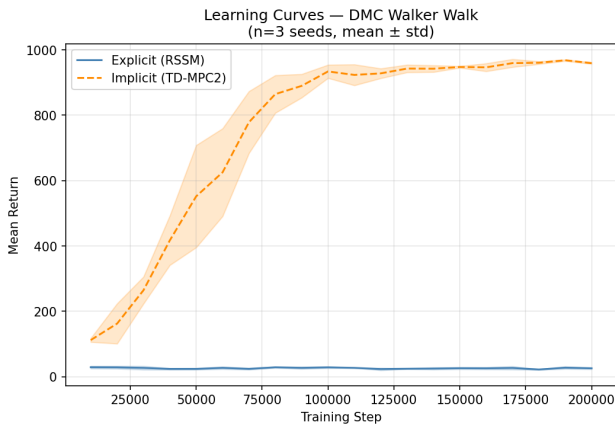


Figure 8. Learning curves on `walker_walk` (3 seeds). The implicit agent achieves near-expert locomotion (≈ 950 return); the explicit agent remains well below 100.

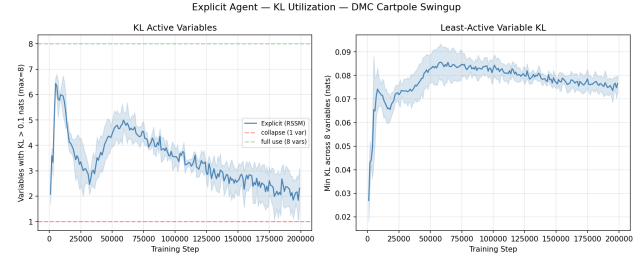


Figure 9. KL utilisation on `cartpole_swingup` (3 seeds). A similar collapse pattern as `LiftCube-v0` is observed, with recovery partially occurring in some seeds.

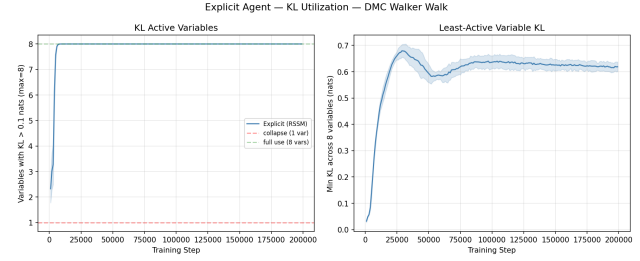


Figure 10. KL utilisation on `walker_walk` (3 seeds). The collapse is consistent; the explicit agent never recovers full variable utilisation on the locomotion task.

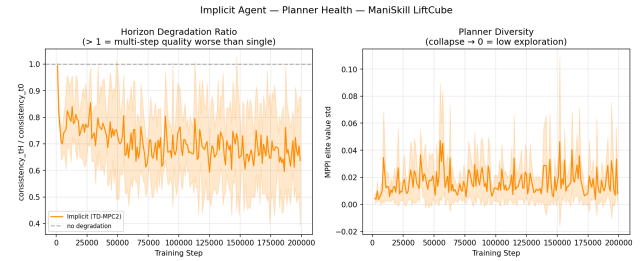


Figure 11. MPPI planner health on `LiftCube-v0` (3 seeds). Top: one-step consistency error (`consistency_t0`). Middle: multi-step consistency error (`consistency_tH`). Bottom: elite-value diversity (`mppi_elite_value_std`). The horizon degradation ratio $tH/t0$ drops below 1 after step 10k (healthy dynamics), and planner diversity remains volatile but non-zero.

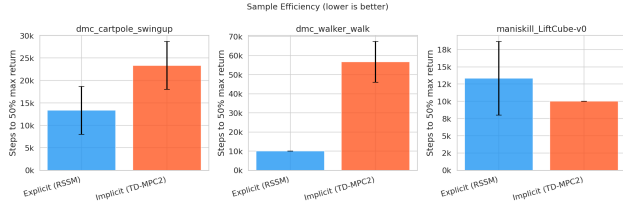


Figure 12. Sample efficiency: steps to reach 50% of each agent’s peak return across all three environments. The implicit agent reaches criterion faster on all tasks.

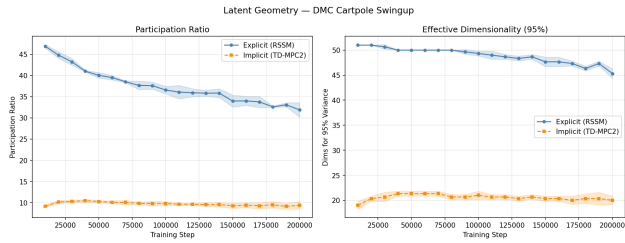


Figure 13. Geometry evolution on cartpole_swingup (3 seeds). The same opposing trends (explicit PR decreasing, implicit PR growing) are observed as on LiftCube-v0.

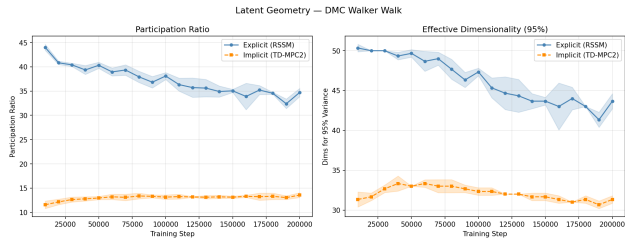


Figure 14. Geometry evolution on walker_walk (3 seeds). The explicit agent’s participation ratio decreases more sharply on this higher-dimensional observation task.